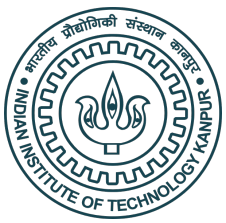
Which Bit Flips Matter?
A Complete Fault-Case Algorithm for Microscaling (MX) Neural Networks

Undergraduate Project (UGP) Final Report
CS496

Peeyush Agarwal
Roll No.: 230749
Department of Aerospace Engineering
Indian Institute of Technology Kanpur

Supervisor: **Prof. Urbi Chatterjee**
Indian Institute of Technology Kanpur

November 2026



Abstract

Microscaling (MX) formats store a block of 32 low-precision values together with one shared 8-bit scale. They make inference cheaper, but they also change what a hardware bit flip does: a flip in an element changes one value, while a flip in the shared scale changes all 32. A fault-injection campaign tells you *how often* such a network fails. It does not tell you *which* flips cause the failures, or what would stop them.

This project develops an algorithm that answers that question for every possible single-bit flip in an MX tensor. A flip becomes a wrong prediction only if five things happen in a row: the bit really changes, the stored number changes, nothing at read time notices, the datapath carries the error, and the top-1 answer moves. The algorithm follows this chain. At its core is a case tree that puts every flip into exactly one of 22 named cases (8 for the scale, 14 for an element). This is checked exhaustively over every code, bit and scale byte of five formats.

On two exhaustive ResNet8 campaigns (1.1 million injections), the case alone explains 91–97% of the variance in damage. Three cases, together 1–2% of all stored bits, cause more than half of all failures, and every one of these catastrophic flips is a $0 \to 1$ flip. Two guards that follow directly from the algorithm, and need no extra memory, cut the failure rate by 4.3× (`e3m2`) and 5.2× (`e4m3`). A new experiment then uses the algorithm as a sampling and protection plan on three networks that have no exhaustive campaign, including a 7-million-parameter RepVGG-A0 and a CIFAR-100 model. With about 1,000 injections it reproduces the exhaustive failure rate of ResNet8 to within 0.2 standard errors, 11–724× more precisely than uniform sampling of the same size. Its predictions hold on every network, and on RepVGG-A0 the same guards cut the failure rate by more than two orders of magnitude.

Contents

1 Introduction **3**
1.1 Research objective . . . 3

2 Background **3**
2.1 Microscaling formats . . . 3
2.2 Two fault sites . . . 4
2.3 Related work and the gap . . . 4
2.4 Metric . . . 4

3 Proposed Methodology: The Flip-Case Algorithm **5**
3.1 Overview . . . 5
3.2 Stage 0: the fault model . . . 5
3.3 Stage 1: the case tree . . . 5
3.4 Stage 2: free checks and guards . . . 7
3.5 Stages 3–5: datapath, decision and use . . . 8
3.6 Using the algorithm to plan a campaign . . . 8

4 Experimental Setup **8**
4.1 Models and data . . . 8
4.2 Experiments . . . 9

5 Results and Analysis **9**
5.1 A few cases hold most of the damage . . . 9
5.2 Flip direction decides the risk . . . 10
5.3 Free checks catch most of the damage . . . 11
5.4 The datapath can turn a catastrophic flip into a mild one . . . 11
5.5 Activations and re-quantisation . . . 11
5.6 New experiment (E29): the algorithm on other networks . . . 12
5.7 Interpretation . . . 14

6 Conclusion **14**

7 Future Work **14**

Acknowledgements **15**

References **15**

1. Introduction

Deep neural network (DNN) accelerators are moving to narrow number formats, because memory traffic and arithmetic both get cheaper as the bit width falls. A 4- to 8-bit number cannot cover the range of a whole weight tensor on its own. *Microscaling* (MX) formats solve this by grouping values into blocks that share one scale factor [1, 2]. The OCP MX specification defines 8-, 6- and 4-bit variants, and they are now supported in commercial hardware.

Hardware is not fault-free. Radiation strikes, voltage droop and ageing can flip a stored bit, and in a DNN such a flip can silently change a prediction. This is called silent data corruption (SDC). DNN reliability under bit flips has been studied for FP32 and integer formats [3, 4, 5]. Those studies assume that each value is its own word, so one flip changes one number. MX breaks that assumption: a flip in the shared scale changes every value in its block.

The first part of this project (reported earlier) built `mxfi`, a bit-accurate fault injector for MX formats, and used it to measure how format, block size, tensor type and bit position affect the failure rate. That work showed *that* formats with equal accuracy can differ greatly in reliability. It could not say exactly *which* flips are responsible, because a campaign returns a single number. To protect a network, a designer needs the second answer, not the first.

1.1. Research objective

> *Can every single-bit flip in an MX network be sorted, before any injection, into a small set of cases that predicts its damage, explains why it happens, and tells us how to stop it?*

The goals of this phase were:

1. Design a complete and unambiguous classification of single-bit flips in MX tensors, covering the scale and the element, and the hardware around them.
2. Measure each case exactly against exhaustive ground truth, and find out how much of the damage the case explains.
3. Turn the algorithm into something usable: cheap checks and guards that remove the dangerous cases, and a sampling plan that cuts the cost of a campaign.
4. Test whether the algorithm carries over to networks it was not built on.

2. Background

2.1. Microscaling formats

An MX block holds $K = 32$ values. It is stored as one 8-bit shared exponent X (the *scale*) and K narrow element codes m_i. The value of element i is

$$v_i = \text{decode}(m_i) \cdot 2^{X-127}, \qquad i = 1, \ldots, K. \tag{1}$$

The scale uses the E8M0 encoding: an unsigned power of two, with code 255 reserved for NaN. Each element is a small sign–exponent–mantissa number. The OCP rule sets

$X = \lfloor \log_2 \max_i |v_i| \rfloor - e_{\max}$, so the largest value of every block lands in the top binade of the element format. Table 1 lists the element formats.

Table 1: MX element formats. "Special codes" are bit patterns that decode to NaN or $\pm\infty$.

MX format	Element	Bits (S/E/M)	Largest value	Special codes
MXFP8	`e4m3`	1/4/3	448	NaN at `S.1111.111`
MXFP8	`e5m2`	1/5/2	57344	$\pm\infty$, NaN (exp. `11111`)
MXFP6	`e3m2`	1/3/2	28	none
MXFP6	`e2m3`	1/2/3	7.5	none
MXFP4	`e2m1`	1/2/1	6	none

2.2. Two fault sites

A single-bit flip can land in one of two places:

- **Element flip.** One code m_i changes, so one value changes.
- **Scale flip.** Flipping bit j of X moves the exponent by exactly $\Delta = \pm 2^j$, so all K values in the block are multiplied by 2^{Δ}. Bit 7 gives $\Delta = \pm 128$.

Scale bits are only $8/(8 + 32w)$ of storage for a w-bit element (about 4% for `e3m2`), but each one has a blast radius of 32 values.

2.3. Related work and the gap

Fault-injection tools such as Ares [4] and PyTorchFI [5] inject random flips and report a failure rate. Statistical fault injection [6] and value-aware sampling such as TreeFI [7] reduce the number of injections needed to estimate that rate. Bit-flip attacks [8, 9] search for the few most damaging bits. None of these gives a *complete, exact* map from a flip to its effect in MX formats, where the scale and the element interact, and where special codes, read-time checks and the datapath decide whether a flip is harmless or catastrophic. That map is what this phase builds.

2.4. Metric

Each campaign runs a fixed set of 200 test images. For one fault f, the damage is the fraction of the 200 predictions that change from the fault-free ("golden") quantised model:

$$d(f) = \frac{1}{200} \sum_{n=1}^{200} \mathbb{1}\left[\hat{y}_n^{(f)} \neq \hat{y}_n^{\text{golden}}\right], \qquad \text{rate} = \mathbb{E}_f\left[d(f)\right]. \tag{2}$$

This is the *per-inference* failure rate: the probability that a random fault changes a random prediction. An image whose output is NaN or ∞ counts as changed. The earlier phase showed that the more common "did any image change" metric depends heavily on which images are used, so it is not used here.

3. Proposed Methodology: The Flip-Case Algorithm

3.1. Overview

A bit flip becomes a wrong answer only if five things all happen (Figure 1). The algorithm checks them in order, and each stage either stops the fault or passes it on.

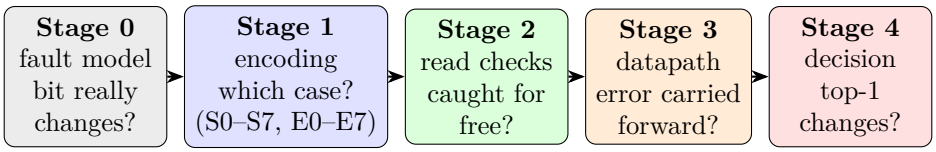


Figure 1: The five-stage chain. A flip that stops at any stage causes no failure. Stage 1 is the case tree; stages 0, 2 and 3 describe the memory, the reader and the arithmetic around it.

3.2. Stage 0: the fault model

A transient flip can go $0 \to 1$ or $1 \to 0$. A stuck-at fault only does something when the stored bit differs from the stuck value. Memory technologies are often asymmetric (for example, DRAM charge loss is mostly $1 \to 0$), so the algorithm weights each flip by the probability of its direction rather than assuming both are equally likely.

3.3. Stage 1: the case tree

This is the core of the algorithm. For a scale flip, $X' = X \oplus 2^j$ and $\Delta = X' - X$. For an element flip, $c' = c \oplus 2^b$ and the value moves from v to v'. The cases are checked in the order of Algorithm 1, and the first match wins. Tables 2 and 3 give their meaning.

Algorithm 1 Stage 1: the case of one single-bit flip

if the flip hits scale byte X, bit j **then**
  $X' \leftarrow X \oplus 2^j$; $\Delta \leftarrow X' - X$ ▷ block scaled by 2^Δ
  if $X' = 255$ **then return S1** to NaN ▷ whole block NaN
  else if $X = 255$ **then return S7** from NaN ▷ only after an earlier fault
  else if every element is ± 0 **then return S0** zero block ▷ masked
  else if $\Delta = +128$ or block overflows FP32 **then return S2** overflow
  else if $\Delta \geq 8$ **then return S3** grow $\geq 256\times$
  else if $\Delta > 0$ **then return S4** grow 2–16×
  else if $\Delta \geq -2$ **then return S5** shrink 2–4×
  else return S6 erase ▷ shrink $\geq 16\times$
  end if
else ▷ element code c, bit b
  $c' \leftarrow c \oplus 2^b$
  if c is NaN/Inf **then return E7** from special
  else if c' is NaN/Inf **then return E1** to special
  else if b is the sign bit **then return E0** if $v = \pm 0$, else **E2** negate
  else if exponent bit set $0 \to 1$ **then return E3a** from zero / **E3b** subnormal→normal / **E3** grow
  else if exponent bit cleared **then return E4a** to zero / **E4b** normal→subnormal / **E4** shrink
  else if mantissa bit set **then return E5a** from zero / **E5** grow
  else return E6a to zero / **E6** shrink
  end if
end if

Table 2: Scale cases. P_B is the block's partial sum in the next layer.

Case	Condition	Effect
S0 zero block	all elements ± 0	none (masked)
S1 to NaN	$X' = 255$	32 NaNs
S2 overflow	$\Delta = +128$ or FP32 overflow	$\pm\infty$ when $\lvert P_B \rvert\, 2^\Delta$ overflows
S3 grow large	$\Delta \geq 8$	block $\times 256$ or more
S4 grow small	$\Delta \in \{1, 2, 4\}$	block $\times 2$, $\times 4$, $\times 16$
S5 shrink small	$\Delta \in \{-1, -2\}$	block $\div 2$, $\div 4$
S6 erase	$\Delta \leq -4$	block $\div 16$ or more, nearly deleted
S7 from NaN	$X = 255$	only after an earlier fault

Table 3: Element cases. Sub-cases mark a change of value class (zero, subnormal, normal).

Case	Field / direction	Effect
E0 masked	sign of ± 0	none
E1 to special	any bit, into NaN/Inf code	NaN or $\pm\infty$ weight
E2 negate	sign	$v \to -v$, error $2\lvert v\rvert$
E3 / E3a / E3b	exponent $0 \to 1$	grow $\times 2^{2^k}$ / from zero / sub.$\to$normal
E4 / E4a / E4b	exponent $1 \to 0$	shrink $\div 2^{2^k}$ / to zero / normal$\to$sub.
E5 / E5a	mantissa $0 \to 1$	small growth / from zero
E6 / E6a	mantissa $1 \to 0$	small shrink / to zero
E7 from special	code already NaN/Inf	only after an earlier fault

Theorem 1 (Completeness)**.** *For each of the five element formats, every pair (code, bit) of an element and every pair (byte $X \in \{0, \ldots, 255\}$, bit j) of a scale, for both zero and non-zero blocks, falls into exactly one case of Algorithm 1.*

Argument and check. Inside each branch the conditions are tested in a fixed order and the last branch has no condition, so every flip gets a case and no flip gets two. Because the input space is finite, this was also checked by brute force: the test suite enumerates all codes, bits and scale bytes of all five formats and asserts that each lands in one valid case (38 tests, all passing). □

The case tree explains the *catastrophic* flips. Call E1, S1 and S2 the catastrophic cases: they produce NaN or ∞ values.

Proposition 1 (Catastrophic flips are $0 \to 1$)**.** *Every flip in a catastrophic case sets a bit that was* 0*.*

Proof. NaN and Inf codes are runs of 1s: `S.1111.111` in `e4m3`, exponent `11111` in `e5m2`, and $255 =$ `11111111` for E8M0. A finite code has at least one 0 in that run, and a single flip can reach the all-ones pattern only by setting that 0. For S2, $\Delta > 0$, and $\Delta = X' - X = +2^j$ exactly when bit j was 0. □

So a memory that mostly loses charge ($1 \to 0$) cannot produce a catastrophic flip at all, and a stuck-at-0 bit is almost harmless. Section 5.2 measures this.

3.4. Stage 2: free checks and guards

A clean OCP MX tensor obeys invariants that can be checked at read time with *no extra memory*:

- **D1:** no element code is NaN/Inf (a clean tensor never stores one). If one is found, guard **G1** decodes it as 0, so E1 becomes "one weight zeroed".
- **D2:** every scale lies inside the tensor's clean band $[X_{\min}, X_{\max}]$, known offline for weights. If a scale is above the band, guard **G2** treats the block as 0, so S1/S2/S3 become an S6-like erase.
- **D3:** every non-zero block has an element in the top binade (the OCP rule guarantees it). A block that lost it was hit by a fault.

3.5. Stages 3–5: datapath, decision and use

Whether a NaN reaches the output depends on the hardware. With IEEE arithmetic NaN survives every operation, and $\text{NaN} \cdot 0 = \text{NaN}$, so even zero inputs do not mask it. A ReLU written as `x > 0 ? x : 0` turns NaN into 0, which erases one output channel instead. $+\infty$ passes any ReLU, $-\infty$ does not. A finite error reaching the next layer is about $\delta \cdot a$ for an element (weight error times input activation) and $(2^{\Delta} - 1)\, P_B$ for a scale. At stage 4 the outcome is one of: no change, silent corruption, or a non-finite output that one `isfinite` check on the logits catches. Stage 5 adds up the damage over all faults, weighted by stage 0.

3.6. Using the algorithm to plan a campaign

Stage 1 needs no injection: the case of every stored bit can be counted exactly from the quantised weights. Element cases depend only on (code, bit), so a histogram of codes per tensor is enough. This makes the algorithm a sampling plan. Let stratum h be the set of faults with case c and direction (bit was 0 or 1), with exact size N_h and weight $W_h = N_h/N$. Then

$$\widehat{\text{rate}} = \sum_h W_h\, \bar{d}_h, \qquad \widehat{\text{Var}} = \sum_h W_h^2\, \frac{s_h^2}{n_h} \left(1 - \frac{n_h}{N_h}\right), \tag{3}$$

where $\bar{d}_h$ and s_h^2 are the mean and variance of the n_h injected faults of stratum h. Masked cases (E0, S0) have damage exactly 0 and are not injected at all. Small strata are injected completely. The catastrophic strata, which hold most of the damage but very few faults, are no longer left to chance.

4. Experimental Setup

4.1. Models and data

Table 4: Networks. Accuracy is the MX-quantised accuracy on the 200 campaign images.

Network	Dataset	Parameters	Used in	Ground truth
ResNet8 (width 16) [10, 11]	CIFAR-10	78 K	E25–E29	exhaustive
ResNet8	CIFAR-100	85 K	E29	sampled
RepVGG-A0 [12]	CIFAR-10	7.0 M	E28, E29	sampled

All weights of all convolutional and linear layers are quantised to MX with block size 32 and the OCP scale rule. CIFAR-100 stands in for a harder task, since ImageNet was not available on the machines used. The two ResNet8 CIFAR-10 networks are the exact checkpoints on which the exhaustive campaigns were recorded (one for `e3m2`, one for `e4m3`).

4.2. Experiments

Table 5: Experiments of this phase.

ID	What it does
E25	Labels every fault of the two exhaustive ResNet8 campaigns (483,904 faults for `e3m2`, 638,624 for `e4m3`) with its case, so each case gets its exact damage.
E26	Re-injects the catastrophic cases under a NaN-zeroing ReLU and under guard G1.
E27	Stage 0 and 2: flip direction, checks D1–D3 and guards, held-out variance explained, protection order.
E28	Activations: case mix, scale-band checks, and how re-quantisation spreads an error (exact counting, no injection).
E29	**New:** the algorithm as a sampling and protection plan on three networks (Section 5.6).

E25, E27 and E28 were re-run for this report and reproduced the earlier outputs exactly. E29 injects 60 faults per stratum on the ResNet8 networks and 25 on RepVGG-A0, and scores each fault three times: with no protection, with guards G1+G2, and with a NaN-zeroing ReLU. Everything ran on a 4-core laptop CPU, except the two exhaustive campaigns, which ran earlier on the IIT Kanpur GPU cluster.

5. Results and Analysis

5.1. A few cases hold most of the damage

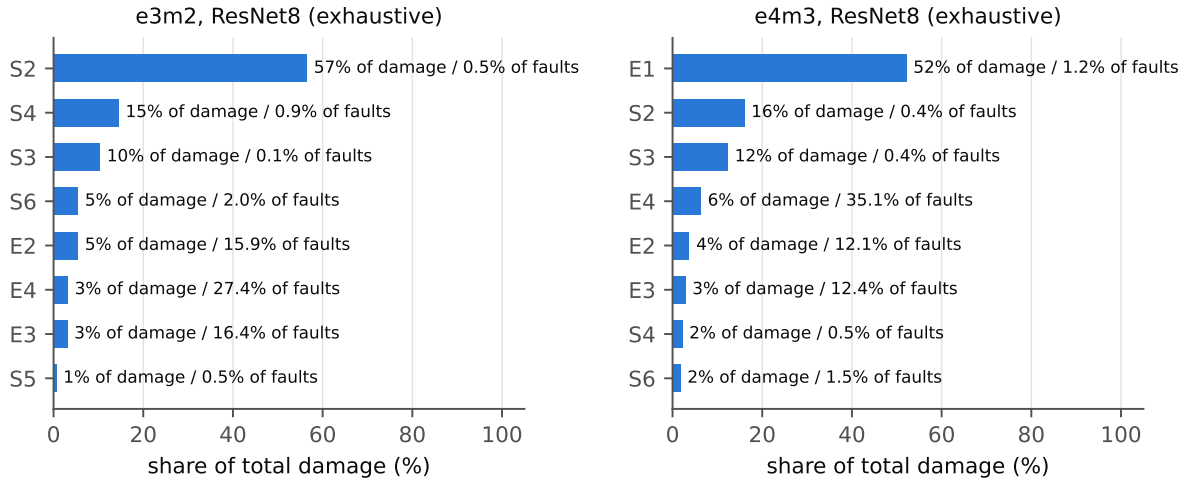


Figure 2: Share of total damage per case on the exhaustive ResNet8 campaigns (top 8 cases). Each bar is labelled with its share of damage and its share of all faults.

Figure 2 shows the main pattern. In `e3m2`, the S2 scale overflow is 0.5% of all faults and causes 57% of all damage (mean damage 0.97: almost every image fails). In `e4m3`, flips

into a NaN code (E1) are 1.2% of faults and cause 52% of damage, with damage exactly 1.0 for every one of them. The common element cases (E2–E6) are 80–95% of faults but only 9–16% of damage.

The case explains the catastrophic part almost completely, but not the small finite errors (Table 6). Within ordinary element flips the case explains only 2–5%; what matters there is the size of the error that reaches the next layer, $|v' - v| \cdot 2^{X-127}$ times the input activity. That raises the explained variance to 64–67%.

Table 6: Held-out variance of damage explained (R^2, two-fold), adding one factor at a time.

	All faults		**Ordinary element flips**	
Model	`e3m2`	`e4m3`	`e3m2`	`e4m3`
Site only (scale or element)	0.22	0.09	0.00	–
+ case (Algorithm 1)	**0.91**	**0.97**	0.02	0.05
+ bit position and layer	0.94	0.98	0.19	0.21
+ error size × input activity	0.94	0.98	**0.67**	**0.64**

5.2. Flip direction decides the risk

Table 7: Stage 0 on the exhaustive campaigns. "0→1 only" is also the rate under stuck-at-1 faults.

	`e3m2`	`e4m3`
Stored bits that are 0	43%	40%
Damage caused by $0 \to 1$ flips	87.5%	88.7%
Rate, $0 \to 1$ flips only	0.00768	0.02089
Rate, $1 \to 0$ flips only	0.00110	0.00267
Ratio	7.0×	7.8×
Catastrophic faults that are $0 \to 1$	2,468 of 2,468	10,321 of 10,321

As Proposition 1 predicts, every catastrophic fault is a $0 \to 1$ flip. The failure rate is therefore not a property of the format alone: a memory whose upsets are mostly $1 \to 0$ would see about 7–8× fewer failures than the usual symmetric fault model assumes.

5.3. Free checks catch most of the damage

Table 8: Stage 2: share of damage flagged by each read-time check, and the rate after the guards.

	`e3m2`	`e4m3`
D1 element code is NaN/Inf	–	52.2%
D2 scale outside its tensor's clean band	85.7%	32.1%
D3 block lost its top-binade element	0.1%	0.2%
D1 + D2 + D3 together	85.8%	84.5%
Rate, no guard	0.00878	0.0236
Rate, G1 + G2	0.00206 (**4.3**× lower)	0.00454 (**5.2**× lower)

The three checks flag only 3.5–4.1% of faults, but those faults carry 85% of the damage. The two guards cost no storage, only a comparison at read time. G1 alone cut the damage of an E1 fault from 1.0 to 0.0064 (measured on 2,000 injected E1 faults).

5.4. The datapath can turn a catastrophic flip into a mild one

Table 9: Stage 3: mean damage of the catastrophic cases under three datapaths (E26).

Case	**ReLU passes NaN**	**ReLU zeroes NaN**	**Read guard**
E1 element → NaN, all layers	1.00	0.085	0.0064 (G1)
E1 element → NaN, final layer	1.00	1.00	0.002–0.02 (G1)
S2 scale +128, `e3m2` / `e4m3`	0.98 / 0.98	0.97 / 0.97	erased (G2)

The same E1 flip is catastrophic or mild depending on one detail of the ReLU. The S2 flip is not helped by the ReLU, because it usually produces $+\infty$, which passes any ReLU; only a scale check stops it.

5.5. Activations and re-quantisation

Activations differ from weights: 40–55% of activation values are exactly 0, so a large share of flips are masked or only turn a 0 into a tiny value. In an all-MX pipeline every activation is re-quantised, and an error that grows one value raises its block's scale and can flush the other 31 values to 0 (Table 10). This reverses the NaN story: the formats without NaN codes are the ones whose errors spread.

Table 10: Share of non-zero neighbours flushed to 0 when the block maximum grows by r (real ResNet8 activations).

Format	$r = 2$	$r = 4$	$r = 16$	$r = 256$	$r = 65536$
`e5m2`	0	0	0	0	0
`e4m3`	0	0	0.01%	0.1%	29%
`e3m2`	0.5%	0.9%	3.9%	55%	100%
`e2m1`	15%	29%	91%	100%	100%

5.6. New experiment (E29): the algorithm on other networks

E25–E28 built the algorithm on one network. E29 asks two practical questions on networks that have no exhaustive campaign: (1) does the case-stratified estimate of Section 3.6 give the right rate with far fewer injections, and (2) do the guards and the direction rule carry over?

Estimation. Table 11 shows the case-stratified estimate. On the two ResNet8 networks, where the exhaustive answer is known, about 1,000 injections give the exhaustive rate to within 0.2 standard errors. An exhaustive campaign needed 484–639 thousand. Compared with uniform sampling of the same size, the variance is 11–724× smaller. Put another way, a uniform campaign would need that many times more injections for the same precision. The RepVGG-A0 fault space has 44–58 million bits, so an exhaustive campaign is out of reach there. The algorithm needs 425–450 injections, and most of them only confirm that the ordinary cases do almost nothing.

Table 11: E29 estimation. "Faults" is the exact size of the fault space, counted by the algorithm without injection. Rate is mean ± one standard error. Variance reduction is against a uniform campaign with the same number of injections.

Network	**Fmt**	**Faults**	**Injected**	**Rate (E29)**	**Exhaustive**	**Var. red.**
ResNet8, C-10	`e3m2`	483,904	1,020	0.00880 ± 0.00038	0.00878	41×
ResNet8, C-10	`e4m3`	638,624	881	0.02347 ± 0.00043	0.02356	108×
ResNet8, C-100	`e3m2`	519,904	1,020	0.0152 ± 0.0008	–	11×
ResNet8, C-100	`e4m3`	686,144	862	0.0251 ± 0.0005	–	75×
ResNet8, C-100	`e5m2`	686,144	724	0.0863 ± 0.0007	–	199×
RepVGG-A0, C-10	`e3m2`	43,980,128	425	$0.00494 \pm 0.00021^*$	–	210×*
RepVGG-A0, C-10	`e4m3`	58,053,504	450	$0.01165 \pm 0.00018^*$	–	724×*

* No injected ordinary element fault changed a prediction, so their variance is estimated as 0 and these two figures are optimistic.

Predictions of the algorithm. Table 12 checks what the algorithm predicts on the new networks.

Table 12: E29: the algorithm's predictions and its guards on every network. "Catastrophic" means cases E1, S1 and S2.

Network	Fmt	Damage from $0 \to 1$	Catastr. faults/damage	Non-finite	G1+G2 cut	NaN-zero ReLU cut
ResNet8, C-10	`e3m2`	89%	0.5% / 57%	56%	4.1×	1.0×
ResNet8, C-10	`e4m3`	88%	1.6% / 69%	69%	5.0×	1.9×
ResNet8, C-100	`e3m2`	77%	0.5% / 33%	33%	1.9×	1.0×
ResNet8, C-100	`e4m3`	86%	1.6% / 63%	63%	4.1×	1.5×
ResNet8, C-100	`e5m2`	95%	7.6% / 88%	88%	12×	1.9×
RepVGG-A0, C-10	`e3m2`	100%	0.5% / 88%	8%	258×*	1.0×
RepVGG-A0, C-10	`e4m3`	100%	1.2% / 95%	73%	1030×*	3.0×

* After the guards, no injected fault outside S3/S4 changed a prediction, so the remaining rate is below what 25 faults per stratum can resolve.

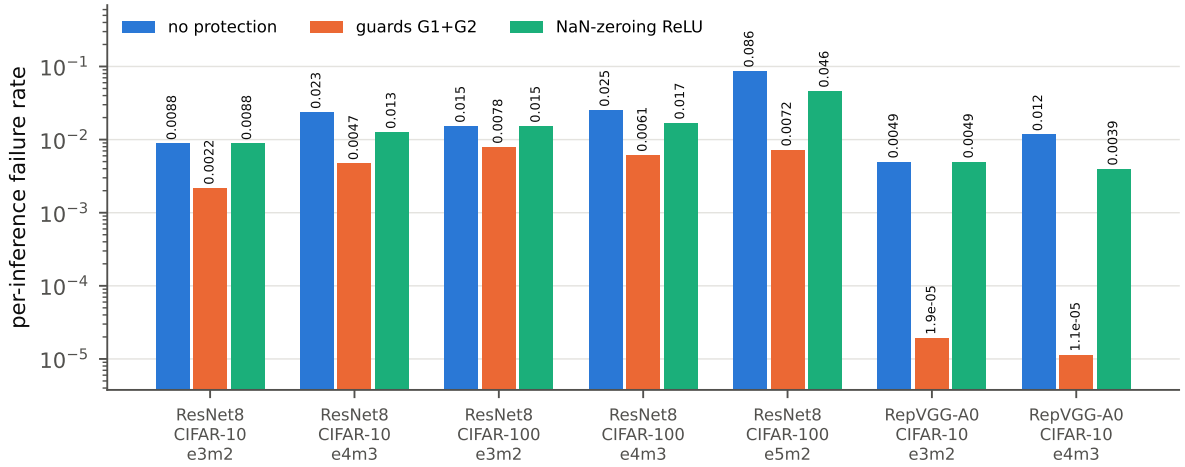


Figure 3: Per-inference failure rate on every network and format (log scale): with no protection, with the read guards G1+G2, and with a NaN-zeroing ReLU instead.

The results confirm the algorithm on every network:

- **Every catastrophic fault is** $0 \to 1$, in all 7 runs, as Proposition 1 says. $0 \to 1$ flips cause 77–100% of all damage.
- **The catastrophic cases dominate, and more so on the larger network.** On RepVGG-A0 they are 0.5–1.2% of faults but 88–95% of damage, and the ordinary element cases did no measurable damage at all. A bigger network absorbs small errors, but not NaN, ∞ or a 2^{128} scale.
- **The guards carry over and help most where they are needed most.** G1+G2 cut the rate 4–12× on ResNet8 and by two to three orders of magnitude on RepVGG-A0. The exception is ResNet8 on CIFAR-100 in `e3m2` (1.9×). That network has low decision margins (55% accuracy), so ordinary finite errors cause a larger share of the damage, and the guards do not target those.
- **The ReLU only helps formats that have NaN codes.** It changes nothing for `e3m2` (no NaN codes; S2 gives $+\infty$, which passes the ReLU), and helps 1.5–3× for `e4m3` and `e5m2`. This is exactly what stage 3 predicts.

- **A catastrophic flip can be silent.** On RepVGG-A0 in `e3m2`, the S2 flips damage 87% of predictions, but only 8% of them give a non-finite output. The weight becomes huge but stays finite, because $|P_B|\,2^{128}$ does not overflow FP32 there (the stage-3 rule). An `isfinite` check on the output misses these, but the scale check D2 catches all of them.

5.7. Interpretation

1. **Most failures come from a tiny, exactly known set of bits.** The catastrophic cases are 1–2% of storage and are found by counting, not by injection.
2. **The case tree is enough for the catastrophic part, but not for small errors.** For those, the size of the error reaching the next layer is what matters.
3. **Every catastrophic flip is $0 \to 1$.** The direction of memory upsets changes the failure rate by 7–8×, so it belongs in the fault model.
4. **Protection should follow the case, not the bit position.** In `e3m2`, scale bit 7 alone is 0.5% of storage and 57% of damage, so protecting a few bits works. In `e4m3`, NaN damage is spread over every element bit and 27% of storage is needed for half the damage; the free guards G1+G2 do far better at zero storage cost.
5. **The algorithm is cheap to use and carries over.** Counting cases needs no injection, and a campaign planned by case needs about a thousand injections instead of hundreds of thousands. Its predictions held on a larger network and a harder dataset, where the catastrophic cases matter even more.

6. Conclusion

This phase of the project turned a fault-injection study of MX networks into an algorithm that says, for every single-bit flip, what it will do and why. The algorithm follows a flip through five stages (fault model, encoding, read checks, datapath, decision), and its core case tree puts every flip into exactly one of 22 cases, which was verified by exhaustive enumeration over five formats.

Measured against 1.1 million exhaustive injections, the case explains 91–97% of the variance in damage. Three catastrophic cases, 1–2% of all bits, cause more than half of all failures, and all of them are $0 \to 1$ flips. Two read-time guards that follow from the algorithm cut the failure rate 4.3–5.2× with no extra memory, and a NaN-zeroing ReLU makes a NaN weight almost harmless.

Used as a campaign plan, the algorithm reproduced the exhaustive ResNet8 rates with about 1,000 injections. On a CIFAR-100 ResNet8 and a 7-million-parameter RepVGG-A0, every prediction held: all catastrophic flips were $0 \to 1$, the catastrophic cases carried 33–95% of the damage, and the free guards cut the failure rate by 1.9× to more than 250×. The practical message is simple: in an MX network, protect the few flips the algorithm names, mainly scale overflows and flips into NaN codes, and the rest matters far less.

7. Future Work

- **Faults outside stored MX values.** Accumulators, partial sums, biases, normalisation parameters and address logic are not covered yet.

- **Accumulating faults.** Weight faults are persistent and can pile up between reloads; only single faults and pairs within one word were measured.
- **Other datapaths, measured end to end.** Saturating overflow, FP16 accumulation and a full MX activation pipeline are stated as rules here, not injected.
- **Larger models and ImageNet.** DeiT and ImageNet were out of reach on the available hardware. The stratified plan of Section 3.6 is what would make such a campaign affordable.
- **Hardware validation** of the guards G1 and G2 on an FPGA or accelerator model.

Acknowledgements

I thank my supervisor, Prof. Urbi Chatterjee, for the problem statement and guidance throughout this project, and IIT Kanpur for access to its GPU cluster.

References

[1] B. D. Rouhani et al., "Microscaling data formats for deep learning," *arXiv:2310.10537*, 2023.

[2] Open Compute Project, "OCP Microscaling Formats (MX) Specification, Version 1.0," 2023.

[3] G. Li, S. K. S. Hari, M. Sullivan, T. Tsai, K. Pattabiraman, J. Emer, and S. W. Keckler, "Understanding error propagation in deep learning neural network (DNN) accelerators and applications," in *Proc. SC*, 2017.

[4] B. Reagen et al., "Ares: A framework for quantifying the resilience of deep neural networks," in *Proc. DAC*, 2018.

[5] A. Mahmoud et al., "PyTorchFI: A runtime perturbation tool for DNNs," in *Proc. DSN Workshops*, 2020.

[6] R. Leveugle, A. Calvez, P. Maistri, and P. Vanhauwaert, "Statistical fault injection: Quantified error and confidence," in *Proc. DATE*, 2009.

[7] TreeFI: value-aware statistical fault injection for DNNs, in *Proc. ICCAD*, 2026.

[8] A. S. Rakin, Z. He, and D. Fan, "Bit-flip attack: Crushing neural network with progressive bit search," in *Proc. ICCV*, 2019.

[9] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitraş, "Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks," in *Proc. USENIX Security*, 2019.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016.

[11] C. Banbury et al., "MLPerf Tiny benchmark," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[12] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proc. CVPR*, 2021.